arXiv:2611.15105v1 [cs.DB] 11 Nov 2026

VecDBLens: An Evaluatology-Based Framework for Vector Database Diagnosis

Yutong Zhou[1,2], Guoxin Kang[1], Wenxin Zhou[1,2]*, Haoyu He[3]*, Yuedong Zhu[4]
Lei Wang[1], Wanling Gao[1], Fanda Fan[1], Jianfeng Zhan[1]

[1]Institute of Computing Technology, Chinese Academy of Sciences
[2]University of Chinese Academy of Sciences
[3]The Chinese University of Hong Kong, Shenzhen
[4]Universiti Sains Malaysia

Abstract

Vector databases are increasingly deployed as retrieval pipelines in which representation, indexing, and query-time execution jointly determine accuracy, throughput, and cost. Existing vector database evaluations primarily focus on indexing while treating embeddings as fixed inputs and ignoring the role of query-time execution, making it difficult to decide whether a bottleneck should be addressed by changing the embedding model, the index, or the query-processing functions. To address the above limitation, we introduce **VecDBLens**, an Evaluatology-based framework that treats the function under diagnosis as an *evaluated object* and all remaining dataset, workload, parameter, and runtime factors as matched *evaluation conditions*. VecDBLens attributes performance differences to individual evaluated objects through comparisons under matched evaluation conditions, summarizing their performance and stability across multiple datasets and workloads. It further analyzes interactions among objects using analysis of variance (ANOVA), thereby turning vector database benchmarking from leaderboard ranking into a reproducible diagnostic procedure. VecDBLens reveals two key insights. First, retrieval performance is fundamentally upper-bounded by embedding quality, revealing that many apparent indexing failures are therefore representation-side limitations. Second, stronger index parameterization is not always beneficial: overly aggressive configurations can reshape neighborhood topology and cause recall regression. These results challenge the common practice of evaluating vector databases primarily through indexing and show that real-world limitations arise from constraints across retrieval functions rather than any single function alone. VecDBLens provides a principled basis for identifying the true performance drivers of vector databases and for guiding their design in practical applications. The source code is available from https://github.com/zhouyutong20/VecDBLens.

1 Introduction

Background. Vector databases have become core infrastructure for modern retrieval applications, including semantic search, recommendation, retrieval-augmented generation, and multimodal retrieval [1, 2, 3, 4, 5]. Their observed performance, however, is not determined by an ANN index in isolation [6, 7, 8, 9, 10, 11]. A deployed vector retrieval pipeline first maps raw objects into a representation space, then organizes vectors into an index, and finally executes query-time search together with runtime constraints.

Limitation and Motivation. Existing benchmarks have produced valuable progress in comparing ANN algorithms, typically under fixed embeddings and a narrow range of configurations [12, 13, 14]. This design is appropriate for measuring approximation loss inside a given vector space, yet it is

*Wenxin Zhou and Haoyu He contributed equally and share third authorship.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Track on Evaluations and Datasets.

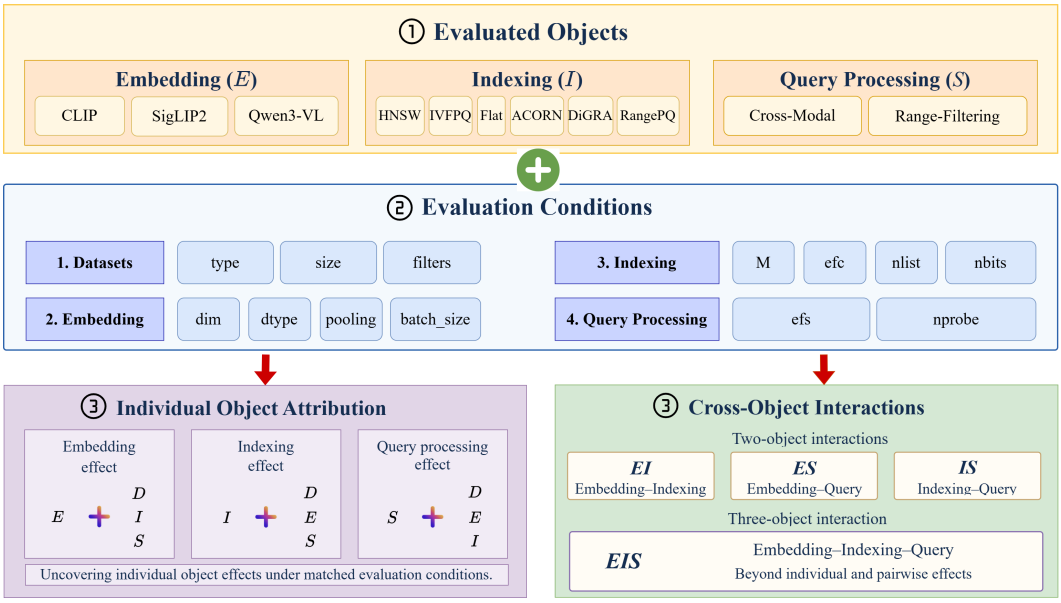

Figure 1: Overview of VecDBLens. VecDBLens first decomposes a vector database into three evaluated objects: embedding, indexing, and query processing, and defines an evaluation condition space covering datasets and object configurations. It then attributes performance differences to individual evaluated objects through comparisons under matched evaluation conditions and analyzes interactions among objects using ANOVA, enabling systematic diagnosis of performance variation and benchmark insights.

insufficient for diagnosing end-to-end performance in real applications. The same low recall can arise from a weak **embedding** that places relevant objects far apart, an **index** that fails to retain the embedding-defined neighborhood, or a **query-processing** strategy that becomes inefficient under filtering or other constraints.

This paper asks a diagnostic question: *when a vector retrieval pipeline fails to meet a target Recall–QPS trade-off, which object is actually responsible?* Answering this question requires evaluating objects under matched evaluation conditions rather than ranking whole databases under a fixed benchmark setting.

We identify two evaluation biases that prevent such a diagnosis: (1) ***index-centric attribution.*** Existing benchmarks often define ground truth using distance-based nearest neighbors, which may deviate from the natural relevance relationships in real-world cross-modal retrieval. As a result, retrieval failures caused by weak or misaligned embeddings may be incorrectly attributed to index structures. (2) ***narrow configuration space.*** Although existing benchmarks vary configurations such as graph connectivity, probing budget, or search depth, they often treat embeddings, workloads, and filtering regimes as fixed. This can favor vector databases whose optimal operating regimes happen to match the benchmark setting, while underestimating systems that are effective under different evaluation conditions.

Contribution. To address these issues, we propose **VecDBLens**, shown in Figure 1. VecDBLens decomposes the retrieval pipeline into **embedding**, **indexing**, and **query processing** functions. For each diagnostic question, the function under diagnosis is the *evaluated object* (EO), while all remaining dataset, workload, parameter, and hardware factors form the *evaluation conditions* (ECs). By comparing EOs only over shared or stratified ECs, VecDBLens estimates individual object effects across matched evaluation conditions instead of conflating them with dataset or workload variation. VecDBLens separates the **individual object effects across matched evaluation conditions** from the extrinsic variation introduced by datasets, workloads, filter selectivity, parameter settings, and runtime environments. We summarize our contributions as follows:

First, we identify two key biases in existing vector database benchmarks: **index-centric attribution** and **narrow configuration space**. We formulate a diagnostic framework for vector retrieval pipelines that distinguishes evaluated objects from their evaluation conditions and supports both **attribution to individual objects** and **analysis of interactions among objects**.

Second, we introduce a reproducible diagnostic protocol that attributes performance differences to individual evaluated objects **under matched evaluation conditions** and analyzes interactions among objects **using analysis of variance (ANOVA)**. This protocol goes beyond leaderboard comparison by explaining whether performance variation is driven by embedding, indexing, or query-time execution.

Finally, we instantiate **VecDBLens** on two complementary workloads: multimodal text-to-image retrieval and filtered ANN search. The experiments show that **embedding quality** imposes an effective upper bound on retrieval performance, and **stronger index parameterization** is not monotonically beneficial.

2 Related Work

2.1 Vector Databases, Applications, and Benchmarks

The architecture of vector databases [4, 2, 15, 16] has fundamentally evolved from basic Approximate Nearest Neighbor (ANN) search [17, 7, 6, 3, 11] to complex databases designed for real-world application demands, such as cross-modal retrieval and filtered ANN search [18, 19, 20, 21, 22]. Early developments predominantly focused on ANN index performance. For instance, **Faiss** [3] integrated various indexing algorithms like **HNSW** [6] and **IVFPQ** [7], while libraries such as **hnswlib** [8] pioneered the standalone, highly optimized implementation of the **HNSW** algorithm.

However, isolated ANN indexes struggle with the inherent complexities of real-world scenarios—specifically, overcoming the embedding blind spots in cross-modal data understanding and addressing the overhead of scalar data processing [22, 23]. To resolve these limitations, modern vector databases have emerged, integrating embedding generation as a retrieval function. **Vector databases** like *Milvus* [2] and *Qdrant* [1], for example, natively integrate **embedding models** such as *CLIP* [24], SigLIP2 [25] and Qwen3-VL [26] to facilitate seamless cross-modal search. Similarly, to efficiently handle filtered ANN search, specialized solutions including *RangePQ* [14], *ACORN* [12], and *DiGRA* [13] have been introduced. These approaches optimize query execution through techniques such as range space partitioning and dynamic adjustment of graph connectivity based on filtering selectivity.

Diverging from existing benchmarks [27, 28, 29] that primarily focus on end-to-end rankings or approximation curves, this paper diagnoses the retrieval pipeline as a set of interacting functions: ***embedding, indexing, and query processing***. The goal is to provide a reproducible diagnostic procedure that identifies whether the limiting factor is the representation space, the index approximation, or the query-time execution.

2.2 Evaluation Methodology

Traditional ANN benchmarks primarily assess index recall and efficiency within a given vector space [27, 29]. Iceberg [30] extends evaluation to downstream tasks, including image classification, face recognition, text retrieval, and recommendation, examining embedding loss, metric misuse, and distribution sensitivity. These studies motivate further diagnosis of how retrieval functions affect application-level performance.

Firstly, they predominantly rely on synthetic workloads to reveal how different index structures, parameter settings, and execution backends affect recall, latency, and system cost [27]. A critical flaw of these synthetic workloads is that the nearest neighbors are purely derived from *mathematical distance metrics*, rather than emerging naturally from real-world applications. Our experiments demonstrate that in real-world cross-modal retrieval scenarios, in the worst cases, a mere **2.04%**[2] of the nearest neighbors calculated via distance metrics correspond to the true semantic nearest neighbors.

Secondly, because existing benchmarks strictly focus on ANN indexing, most existing evaluations treat embeddings as *fixed* input vectors and define exact nearest neighbors within the given vector space as the ground truth by *brute-force search* [10, 27, 29, 11]. As a result, they can explain

[2]On VisualNews, using SigLIP2 embeddings, we perform exhaustive Euclidean search with Faiss CPU `IndexFlatL2` over 1,259,732 candidate images for 20,000 caption queries, with both image and text embeddings L2-normalized. A query is counted as a hit if its nearest image is the paired image specified by the dataset. There are 408 hits, yielding Recall@1 = $408/20{,}000 = 2.04\%$.

how much performance is lost during approximate search, but they usually cannot further *attribute end-to-end retrieval performance* to different pipeline functions.

To overcome these limitations, we introduce VecDBLens, a framework that builds on the pipeline functions decomposition and attribution methods of Evaluatology [31, 32], which has been applied to the evaluation of CPUs [33, 34], large language models [35], and time-series forecasting models [36]. By systematically sampling across an extensive configuration space of datasets and function parameters to construct rigorous evaluation conditions, VecDBLens quantifies both average effects and stability of distinct functions, making it possible to distinguish consistent function behavior across conditions from condition-specific wins.

3 The VecDBLens Framework

Existing benchmarks primarily evaluate indexes under fixed embeddings and a narrow range of configurations. Distance-based neighbors can differ from application-level relevance, while different retrieval configurations can perform best under different workloads. These limitations make it difficult to determine whether a retrieval bottleneck originates from embedding, indexing, or query processing. VecDBLens addresses this diagnostic question by attributing performance differences to individual evaluated objects under matched evaluation conditions and quantifying interactions among objects using ANOVA [37].

3.1 Evaluation System Modeling

Problem statement. VecDBLens decomposes the retrieval pipeline into embedding, indexing, and query processing. For a dataset instance d, let $\mathrm{corpus}(d)$ denote its retrieval corpus and $\mathcal{Q}_d$ its query set. Given a query $q \in \mathcal{Q}_d$, the pipeline is

$$\mathcal{R}_q^{e,a,\beta,\gamma} = S_\gamma \left(I_{a,\beta} \left(E_e(\mathrm{corpus}(d)) \right), E_e(q) \right). \tag{1}$$

The embedding function E_e maps raw objects into dense vectors $E_e(\mathrm{corpus}(d))$ and $E_e(q)$ [26, 24]. The indexing function $I_{a,\beta}$ constructs a searchable structure $I_{a,\beta}(E_e(\mathrm{corpus}(d)))$ using algorithm a and construction parameters β, such as HNSW [6], IVFPQ [7], or CAGRA [9]. The query-processing function S_γ searches the index under query-time parameters γ, trading off recall and efficiency. Flat search provides an exact-search reference for the selected embedding and similarity function.

Evaluated objects and evaluation conditions. Guided by Evaluatology [32, 34, 33, 35, 36], we construct a **Self-Contained System (SCS)** for the systematic evaluation of vector databases. The SCS comprises evaluated objects (EOs) and evaluation conditions (ECs).

The evaluated object, denoted by eo, is the model, algorithm, or configuration under diagnosis. An evaluation condition instance, denoted by ec, specifies the values of the remaining factors for that evaluation. A factor selected as the EO in one analysis can therefore be held fixed as part of ec in another. As shown in Figure 1, when the **embedding model** is the EO, the dataset, index configuration, search settings, and runtime environment are ECs. When the **index algorithm** is the EO, the embedding becomes an EC. The full notation and condition-space definition are provided in Appendix A.

3.2 Individual Object Attribution and Cross-object Interactions

Based on the SCS model, VecDBLens first systematically evaluates an object's performance across matched evaluation conditions, then examines how its effect depends on other evaluated objects.

Individual object attribution. VecDBLens constructs the EC space from the Cartesian product of the selected factor values [38], retaining compatible configurations. Each point in this space represents one evaluation condition instance ec. For individual object attribution, VecDBLens varies eo while holding ec fixed. For indexing, for example, ACORN [12] and RangePQ [14] are compared under identical embedded data, queries, search settings, and runtime resources. The evaluated differences in recall, throughput, or cost quantify the performance effect attributable to the EO choice under the matched evaluation condition. VecDBLens aggregates these differences with equal weighting across dataset or workload strata, and reports mean performance and within-stratum variance to characterize effectiveness and stability.

Cross-object interactions. Vector embedding, index construction, and query processing interact. VecDBLens distinguishes these objects for attribution without assuming independence. All evaluation results come from complete end-to-end retrieval pipelines with compatible configurations of all three objects. When one object is selected as the evaluated object (EO), the other two form part of the matched evaluation conditions (ECs). VecDBLens examines these interactions by testing whether the performance difference attributed to one object changes with the configuration of another object. For example, repeating the construction comparison at different search budgets reveals whether more extensive traversal reduces the recall difference between constructed indexes.

Within each index family, VecDBLens evaluates complete matched grids of embedding, construction, and query-processing configurations, fixing the dataset, candidate pool, queries, and runtime environment. Functional ANOVA [37] decomposes Retention@10 and $\log_2(\mathrm{QPS})$ into main effects and two- and three-factor interactions. Main effects summarize each factor's average effect across the other configurations; interaction terms capture departures from their additive combination. Their variance shares quantify interaction strength, while conditional comparisons explain its direction. Formal attribution metrics, grid weights, and interaction definitions are provided in Appendix A.1.

4 Experiments

This section instantiates VecDBLens on two representative scenarios: (1) **Multimodal retrieval scenario (In Sec. 4.2)**. We diagnose how much retrieval quality is determined before indexing and how much of this fixed-representation potential is retained by ANN backends. (2) **Filtered ANN search scenario (In Sec. 4.3)**. We diagnose whether stronger index parameterization consistently improves performance, and how workload conditions, such as filter ratio and dimensionality, reshape the useful regime of different methods.

Statistical protocol details. For each EO, VecDBLens summarizes performance across evaluated ECs. We report **mean performance**, **stability** measured by average within-stratum variance, **bootstrap confidence intervals**, and **observed maximum and minimum values**. Direct comparisons between EOs use matched evaluation conditions where available. For throughput, we report **geometric mean QPS** because throughput varies by orders of magnitude.

4.1 Evaluation Setup

We instantiate the **VecDBLens** Evaluated Object (EO) and Evaluation Condition (EC) framework across two representative scenarios: multimodal retrieval and filtered ANN search. These scenarios provide complementary perspectives on ANN evaluation: cross-modal retrieval examines semantic alignment between queries and retrieved objects, while filtered ANN examines interactions between scalar constraints and ANN indexes. Both have practical relevance to multimedia search, multimodal RAG, and vector queries with scalar constraints. Runtime settings and parameter ranges are provided in Appendix B.

Multimodal Retrieval. The configuration space is defined as $\Theta_{\mathrm{mm}} = \mathcal{E} \times \mathcal{A} \times \mathcal{B} \times \Gamma$, representing embeddings, ANN backends, construction parameters, and search parameters. For embedding attribution, the EO is an embedding model $e \in \{\mathrm{CLIP}, \mathrm{SigLIP2}, \mathrm{Qwen3\text{-}VL}\}$ [24, 25, 26], evaluated against a Flat exact-search EC to expose the representational upper bound.

For ANN attribution, the EO is a backend $a \in \{\mathrm{Faiss\ HNSW}, \mathrm{Faiss\ IVFPQ}, \mathrm{cuVS\ IVFPQ}, \mathrm{cuVS\ CAGRA}\}$ [3, 7, 9] along with its parameters (β, γ), while the selected embedding becomes an EC. Retention is measured relative to the corresponding Flat reference. The data context spans text-to-image retrieval tasks across Flickr30K (test5k/full), MSCOCO (test5k/full), and VisualNews [39, 40, 41].

Filtered ANN Search. The configuration space is $\Theta_{\mathrm{filter}} = \mathcal{A}_f \times \mathcal{B}_f \times \Gamma_f \times \mathcal{T}$, representing algorithms, construction parameters, search parameters, and runtime settings. The evaluated EOs include Flat, RangePQ, DiGRA, ACORN-1, and ACORN-28 [14, 13, 12]. Algorithm-level comparisons use matched thread settings; ACORN is tested under single- and multi-threaded settings with matched construction and search configurations to assess parallel execution effects. For parameter attribution, the EO is a controlled parameter change, such as increasing M or `efSearch`, with the remaining factors fixed as ECs. Parameters linked by a configuration rule are changed jointly. The experiments use million-scale workloads derived from the widely used SIFT1M and GIST1M bench-

Table 1: Flat exact-search Recall@10 across embedding models and dataset-pool conditions. Mean and sample variance are computed across five conditions. The 95% CI for the mean is estimated by condition-level bootstrap.

Embedding	Flickr30K test5k	Flickr30K full-pool	MSCOCO test5k	MSCOCO full-pool	VisualNews	Mean	Variance	95% CI
Qwen3-VL	0.9732	0.7562	0.8960	0.5998	0.2687	0.6988	0.0781	[0.4637, 0.8989]
CLIP	0.9008	0.4946	0.6727	0.2973	0.2496	0.5230	0.0729	[0.3177, 0.7345]
SigLIP2	0.5982	0.2246	0.1685	0.0488	0.0724	0.2225	0.0492	[0.0822, 0.4183]

marks [7, 42, 43], with dimensionalities dim $\in \{128, 960\}$ and scalar attributes for controlled range filtering. Unlike cross-modal retrieval, the main challenge is index traversal under scalar constraints. Filtering preserves the distance function, so exact neighbors within the eligible set provide ground truth for approximation accuracy. We evaluate filter ratios $r \in \{0.01\%, 0.1\%, 1\%, 10\%, 50\%\}$ and use Flat exact pre-filtering as the reference for each workload condition.

To extend VecDBLens to a broader range of languages and representation formats, we additionally evaluate Chinese text retrieval on MIRACL and multi-vector retrieval on SciFact and FiQA, with experimental settings and results provided in Appendix E.

4.2 Experiment 1: Embedding Quality and ANN Retention

This experiment applies a two-level attribution. First, we treat the embedding model instance e as the EO and the dataset-pool setting d as the EC, using Flat search to measure retrieval quality under exact search. Second, holding the embedding e and dataset-pool setting d fixed, we treat the ANN backend and its configuration (a, β, γ) as the EO and compare its recall with the corresponding Flat reference, separating representation quality from changes introduced by indexing and query processing.

4.2.1 Embedding-induced upper bound.

For a fixed embedding model instance e and dataset instance d, Flat exact search yields the best attainable retrieval quality:

$$U(e, d, k) = R_k(e, \text{Flat}, \varnothing, \varnothing, d). \tag{2}$$

Rather than replacing ground-truth labels, $U(e, d, k)$ quantifies the intrinsic geometric alignment of relevant objects in the representation space. If an embedding space maps relevant pairs far apart, downstream ANN indexes cannot systematically recover this lost relevance.

As shown in Table 1, Qwen3-VL achieves the highest mean Flat Recall@10 across five conditions (0.6988), outperforming CLIP (0.5230) and SigLIP2 (0.2225). This indicates that a large portion of retrieval variation is predetermined before indexing. Many observed failures are fundamentally representation-side limitations.

To distinguish embedding-model effects from differences in output dimensionality and assess the uncertainty of matched-condition comparisons, we conduct matched-dimension experiments and cluster-aware significance tests (Appendix C.1).

> **Insight 1.** ***Embedding quality establishes the strict upper bound for retrieval performance.***

4.2.2 Candidate-pool scaling.

We examine whether exact-search retrieval quality is robust to distractor scaling by expanding the candidate pool from `test5k` to `full-pool`. We define Pool-Scaling Retention as $\text{PSR}(e, d_{\text{test5k}}, d_{\text{full}}, k) = U(e, d_{\text{full}}, k)/U(e, d_{\text{test5k}}, k)$, which measures the retention of exact-search recall under larger pools. We macro-average PSR across Flickr30K and MSCOCO.

Figure 2 shows that Qwen3-VL retains a macro-averaged 72.3% of its Recall@10 after expansion, compared with CLIP (49.6%) and SigLIP2 (33.2%). Stronger embeddings achieve higher fixed-pool recall and exhibit greater robustness to candidate-pool expansion in these experiments. Detailed numerical results are reported in Appendix C.

> **Insight 2.** ***High-quality embeddings preserve more of their exact-search recall under candidate-pool expansion.***

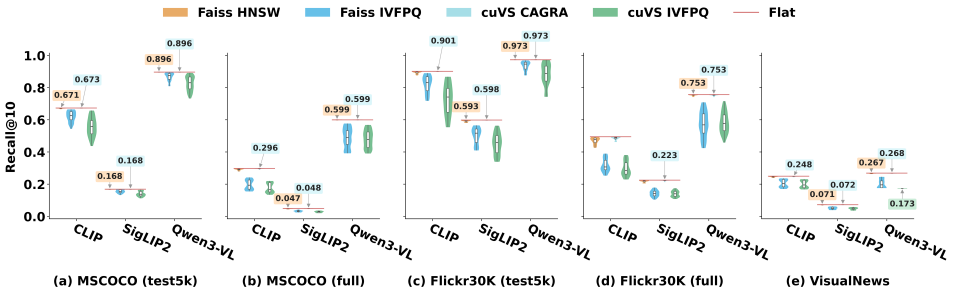

Figure 2: Recall@10 distributions on MSCOCO, Flickr30K, and VisualNews.

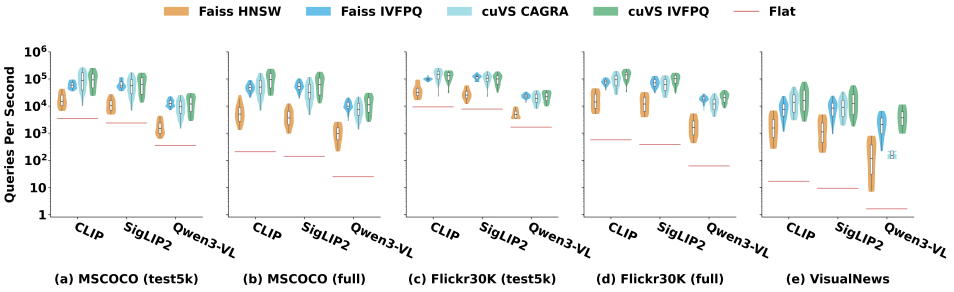

Figure 3: QPS distributions on MSCOCO, Flickr30K, and VisualNews.

4.2.3 ANN retention relative to Flat.

We next quantify how much of the embedding's exact-search recall is retained by the ANN backend. Under matched ECs, we define $\mathrm{Gap}(e, a, \beta, \gamma, d, k) = U(e, d, k) - R_k(e, a, \beta, \gamma, d)$ and $\mathrm{Retention}(e, a, \beta, \gamma, d, k) = R_k(e, a, \beta, \gamma, d)/U(e, d, k)$ for $U(e, d, k) > 0$. A Retention score near 1 indicates that ANN recall closely matches the corresponding Flat reference.

Figure 2 shows that graph-based indexes are highly effective at retaining exact-search recall: cuVS CAGRA and Faiss HNSW achieve mean Retention@10 values of 0.9959 and 0.9875, respectively, across all configurations. In contrast, PQ-based indexes trade recall for memory efficiency, with Faiss IVFPQ dropping to 0.7932. Detailed retention and Gap-to-Flat statistics are provided in Appendix C.

> **Insight 3.** ***ANN indexes may lose relevant results through approximation but cannot improve the underlying embedding's semantic alignment.***

4.2.4 Recall-QPS frontier

VecDBLens further evaluates how embeddings and indexes jointly shape the Pareto frontier of retrieval quality and system cost. While Qwen3-VL provides the highest exact-search recall, searching its 4096-dimensional vectors incurs substantially lower throughput. Under Flat search, Qwen3-VL is $5.6\times$–$10.0\times$ slower than CLIP across dataset conditions.

However, ANN indexing substantially reduces the search cost while retaining retrieval quality. As shown in Figure 3, among configurations with $\mathrm{Retention@10} \geq 0.99$, Faiss HNSW boosts QPS from 13.8 (Flat) to 1898.6 on VisualNews with CLIP. GPU-accelerated CAGRA reaches 29,433.6 QPS while retaining 99.28% of the corresponding Flat recall.

Ultimately, end-to-end performance depends on interactions among retrieval functions. The embedding function determines exact-search recall and influences distance-computation cost, while the indexing and query-processing functions determine the recall-throughput trade-off relative to this reference.

> **Insight 4.** ***The Recall-QPS Pareto frontier is jointly determined: embeddings establish the exact-search reference, and indexing and query processing determine how efficiently its recall is retained.***

Table 2: Interaction shares of within-condition variance (%) across five evaluation conditions. Ranges cover three build repetitions, with each run analyzed separately. E, I, and S denote embedding choice, index construction, and query processing, respectively.

	Retention@10		$\log_2$(QPS)	
Interaction	**Faiss HNSW**	**Faiss IVFPQ**	**Faiss HNSW**	**Faiss IVFPQ**
$E \times I$	1.19–17.89	0.74–2.71	0.26–0.33	0.34–1.58
$E \times S$	3.66–16.89	6.72–6.99	0.23–0.97	0.82–1.16
$I \times S$	0.30–1.69	0.17–0.23	0.03–0.20	0.56–0.80
$E \times I \times S$	0.16–1.02	0.14–0.20	0.07–0.26	0.10–0.43
Total interactions	19.34–22.01	7.89–10.03	0.67–1.75	1.85–3.96

4.2.5 Cross-object interactions

We examine how the performance effect of one evaluated object depends on other objects using functional ANOVA (Section 3.2). Table 2 shows that interactions explain a larger share of Retention@10 variation than of throughput variation. Embedding–query processing dominates IVFPQ's quality interactions. For HNSW, the dominant contribution varies between embedding–index construction and embedding–query processing across build repetitions. Three-factor interactions remain small for both indexes.

Conditional comparisons illustrate how query processing moderates construction effects. Increasing HNSW's `efSearch` by $8\times$ reduces the mean standard deviation of Retention@10 across construction configurations by 36.0%–78.0%, showing that larger search budgets can reduce sensitivity to index construction.

> **Insight 5.** ***Cross-object interactions explain a larger share of variation in retrieval quality than in throughput.***

4.3 Experiment 2: Parameterization Attribution in Filtered ANN

This experiment investigates filtered ANN from three attribution perspectives. For algorithm-level comparisons, the EO is the method a, while parameters, filter ratio, dimensionality, and runtime settings are ECs. For parameter monotonicity analysis, the EO is a controlled parameter change (e.g., M or `efConstruction`), with the remaining factors fixed as matched ECs. Parameters linked by a configuration rule are changed jointly. For workload analysis, filter ratio and dimensionality define EC strata for comparing exact pre-filtering with approximate search.

4.3.1 Non-monotonicity of stronger parameterization

We compare adjacent parameter levels under matched ECs and record a violation whenever Recall@100 decreases. Table 3 shows that both ACORN and DiGRA exhibit such decreases as construction parameters increase.

We identify two mechanisms that can reduce filtered recall in the evaluated ACORN implementation. First, increasing graph connectivity raises the threshold for exploring neighbors of neighbors. As Table 4 shows, nodes in the denser graph have more filter-compatible neighbors on average, but fewer nodes qualify for two-hop expansion and Recall@100 falls.

Second, a larger construction budget can allow the candidate set to reach a stopping threshold before closer candidates replace those encountered earlier. The resulting graph has longer, less directionally diverse edges. Query traces show that search on this graph visits more nodes but reaches fewer exact neighbors.

We also observe recall decreases under stronger ACORN construction settings on App Reviews and in both CC-News time windows. Appendix D.1 reports the real-workload results.

> **Insight 6.** ***Stronger index construction parameterization is not monotonically beneficial; aggressive graph connectivity can disrupt filtered neighborhoods and regress recall.***

Table 3: Matched-condition monotonicity audit for filtered ANN parameterization. Each comparison increases the selected parameter while keeping other settings fixed, except for parameters coupled by a configuration rule. A violation is recorded when Recall@100 decreases.

Parameter change	Comparisons	Violations	Rate	Max drop
ACORN: M	2244	273	12.2%	0.4332
ACORN: `efConstruction`	1683	306	18.2%	0.5237
ACORN: γ_{ACORN}	1836	762	41.5%	0.0626
DiGRA: M	520	133	25.6%	0.010891

Table 4: Effect of increased graph connectivity on ACORN under matched evaluation conditions.

Measure	Baseline graph	Denser graph
Mean number of filter-compatible neighbors	0.1596	0.2002
Nodes eligible for two-hop expansion (%)	59.97	18.78
Recall@100	0.3150	0.1827

4.3.2 Effect of filter ratio and dimensionality

We examine how filter ratio and dimensionality affect the choice between exact pre-filtering and approximate search. On the 128-dimensional SIFT workload (Figure 4), Flat exact pre-filtering remains competitive at filter ratios of 0.01% and 0.1% because few vectors satisfy the filter. At ratios of 1% and above, Flat must scan a larger candidate set, and DiGRA achieves the highest single-threaded QPS at a target Recall@100 of $\tau = 0.95$.

On the 960-dimensional GIST workload (Figure 5), each distance computation is more expensive. DiGRA achieves the highest single-threaded QPS at the same recall target across all tested filter ratios, including 0.01%. For filter ratios from 1% to 50%, ACORN-28 achieves $10.8\times$–$1051.7\times$ the QPS of single-threaded Flat on SIFT and $97.4\times$–$1842.9\times$ on GIST.

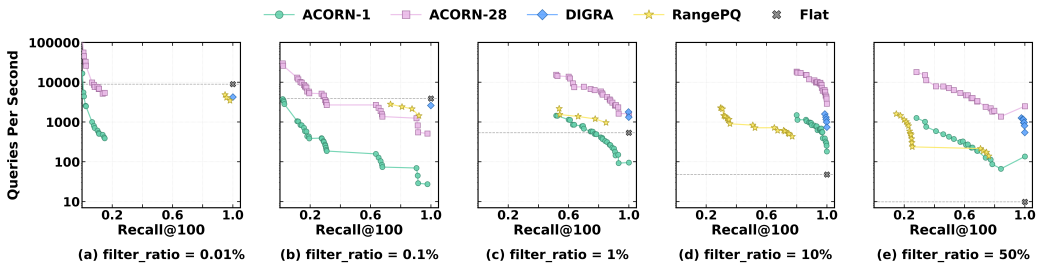

Figure 4: Recall-QPS Pareto frontiers across various filter ratios (128-dimensional data).

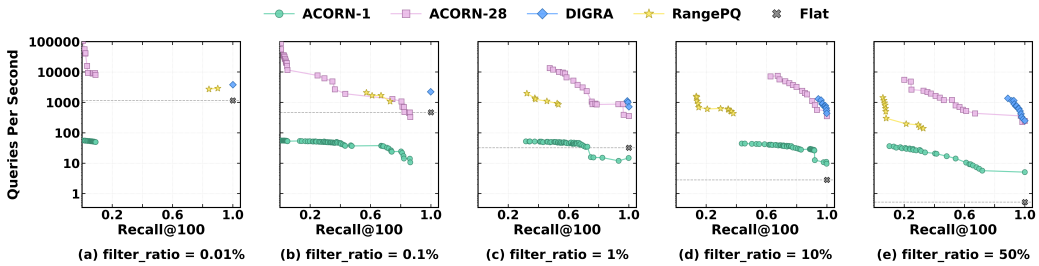

Figure 5: Recall-QPS Pareto frontiers across various filter ratios (960-dimensional data).

Insight 7. ***The boundary between exact pre-filtering and approximate search is non-static; it shifts substantially based on the joint effect of vector dimensionality and filter selectivity.***

4.3.3 Runtime and cost regimes

Beyond recall and throughput, practical deployment depends on index construction time and memory use. Figure 6 shows how these costs vary across filtered ANN methods. DiGRA achieves near-perfect recall in many configurations but requires long build times and substantial memory. RangePQ builds quickly and has the smallest increase in memory use during construction, but none of its tested configurations reaches the Recall@100 target of 0.95 on 960-dimensional GIST. Meanwhile, ACORN strikes a balance, achieving robust Recall-QPS trade-offs—especially when multi-threading (ACORN-28) yields a $14.2\times$ median QPS speedup. Consequently, filtered ANN performance cannot be abstracted to a single metric. It requires object-level diagnosis across recall, throughput, build time, and parameter stability.

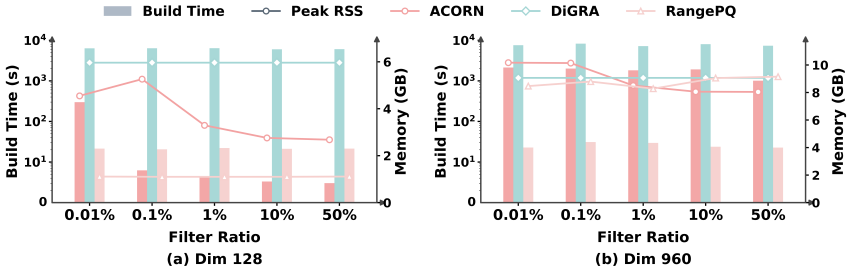

Figure 6: Construction time and memory use of filtered ANN methods across filter ratios and dimensionalities.

> **Insight 8.** ***Distinct algorithmic cost regimes reshape the attainable Pareto frontier; optimal algorithm selection must balance memory, build time, and multi-threading efficiency.***

5 Conclusion

This paper introduces **VecDBLens**, an Evaluatology-based diagnostic framework that shifts vector database retrieval pipeline evaluation from **black-box database comparison** to **object attribution** across ***embedding, indexing, and query processing*** functions. Our analyses yield two critical insights. First, the embedding space strictly defines the upper bound of retrieval performance; downstream ANN indexes only dictate the efficiency-cost trade-off in realizing this potential, rather than creating semantic relevance. Second, in filtered search, stronger index parameterization is not monotonically beneficial and can surprisingly regress recall. Furthermore, the optimal boundary between exact pre-filtering and ANN approximation is non-static, jointly governed by dimensionality and filter selectivity. Ultimately, by isolating bottlenecks to representation limits, index approximations, or query executions, VecDBLens prevents misdirected system optimizations and guides more robust vector database design.

6 Limitations and Future Work

VecDBLens currently emphasizes recall, throughput, memory usage, construction cost, and parameter stability. These metrics characterize retrieval quality and system cost but provide limited insight into semantic failure mechanisms. Future work will develop finer-grained semantic diagnostics and incorporate energy efficiency and incremental indexing costs.

Beyond cross-modal and filtered retrieval, we have extended evaluation to Chinese text retrieval and multi-vector retrieval in scientific and financial domains (Appendix E). Broader language coverage, temporal data streams, and dynamic workloads remain directions for further evaluation. Extending attribution to database functions such as update processing and query optimization would also broaden the framework's scope.

We are committed to long-term maintenance, including continued updates to the implementation, evaluation protocols, and workload coverage as vector retrieval evolves. Our goal is to advance evaluation from asking which system ranks first to identifying which retrieval function limits performance, under which conditions, and why. This perspective supports reproducible research, efficient system development, and application-aware vector database design.

Acknowledgments and Disclosures

We thank the anonymous reviewers for carefully reading our manuscript and providing constructive feedback that helped improve the clarity and presentation of this paper. This work was supported by the Innovation Funding of the Institute of Computing Technology, Chinese Academy of Sciences (ICT, CAS), under Grant No. E661110.

References

[1] Qdrant Team. Qdrant: Open source vector database and search engine. https://github.com/qdrant/qdrant, 2021.

[2] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 international conference on management of data*, pages 2614–2627, 2021.

[3] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[4] James Jie Pan, Jianguo Wang, and Guoliang Li. Survey of vector database management systems. *arXiv preprint arXiv:2310.14021*, 2023.

[5] Yunfan Gao. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

[6] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[7] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[8] Yury Malkov et al. hnswlib – header-only C++/python library for fast approximate nearest neighbors. https://github.com/nmslib/hnswlib, 2018.

[9] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024.

[10] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. Approximate nearest neighbor search on high dimensional data — experiments, analyses, and improvement. *IEEE Transactions on Knowledge and Data Engineering*, 32(8):1475–1488, 2019.

[11] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search, 2021.

[12] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proc. ACM Manag. Data*, 2(3), 2024.

[13] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibo Wang. DIGRA: A dynamic graph indexing for approximate nearest neighbor search with range filter. *Proc. ACM Manag. Data*, 2025.

[14] Fangyuan Zhang, Mengxu Jiang, Guanhao Hou, Jieming Shi, Hua Fan, Wenchao Zhou, Feifei Li, and Sibo Wang. Efficient dynamic indexing for range filtered approximate nearest neighbor search. *Proc. ACM Manag. Data*, 3(3), June 2025.

[15] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, et al. Manu: a cloud native vector database management system. *arXiv preprint arXiv:2206.13843*, 2022.

[16] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu, Szu-Po Wang, Eric Hanson, Zhou Sun, Robert Walzer, and Jianguo Wang. Singlestore-v: An integrated vector database system in singlestore. *Proceedings of the VLDB Endowment*, 17(12), 2024.

[17] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*, pages 604–613, 1998.

[18] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. Serf: Segment graph for range-filtering approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(1):1–26, 2024.

[19] Joshua Engels, Benjamin Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. Approximate nearest neighbor search with window filters. 2024.

[20] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. irangegraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(6):1–26, 2024.

[21] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. Unify: Unified index for range filtered approximate nearest neighbors search. *arXiv preprint arXiv:2412.02448*, 2024.

[22] Ziqi Wang, Jingzhe Zhang, and Wei Hu. Wow: A window-to-window incremental index for range-filtering approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 3(6):1–27, 2025.

[23] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, et al. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*, pages 3406–3416, 2023.

[24] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763, 2021.

[25] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, et al. SigLIP 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025.

[26] Shuai Bai et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[27] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2020.

[28] Guoxin Kang, Zhongxin Ge, Jingpei Hu, Xueya Zhang, Lei Wang, and Jianfeng Zhan. Bigvectorbench: Heterogeneous data embedding and compound queries are essential in evaluating vector databases. *Proceedings of the VLDB Endowment*, 18(5):1536–1550, 2025.

[29] Harsha Vardhan Simhadri. big-ann-benchmarks: Framework for evaluating ANNS algorithms on billion scale datasets., 2023.

[30] Tingyang Chen, Cong Fu, Jiahua Wu, Haotian Wu, Hua Fan, Xiangyu Ke, Yunjun Gao, Yabo Ni, and Anxiang Zeng. Reveal hidden pitfalls and navigate next generation of vector similarity search from task-centric views: [experiments & analysis]. *Proc. ACM Manag. Data*, 4(1), April 2026.

[31] Jianfeng Zhan, Lei Wang, Wanling Gao, Hongxiao Li, Chenxi Wang, Yunyou Huang, Yatao Li, Zhengxin Yang, Guoxin Kang, Chunjie Luo, et al. Evaluatology: The science and engineering of evaluation. *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, 4(1):100162, 2024.

[32] Jianfeng Zhan, Lei Wang, Wanling Gao, Chenxi Wang, Hongxiao Li, Fanda Fan, and Guoxin Kang. *Evaluatology: The Science of Uncovering the Effects*. BenchCouncil Press, Hong Kong, version 1.0 (first edition, 2025) edition, 2025.

[33] Chenxi Wang, Lei Wang, Wanling Gao, Fanda Fan, Yuchen Su, Yutong Zhou, Yikang Yang, and Jianfeng Zhan. Achieving consistent and comparable CPU evaluation. *arXiv preprint arXiv:2411.08494*, 2025.

[34] Chenxi Wang, Yuchen Su, Lei Wang, Guoxin Kang, Wanling Gao, Fan Zhang, and Jianfeng Zhan. Poster: Benchcpu: Evaluatology-based cpu benchmarking. 2026.

[35] Zhengshuyuan Tian, Wanling Gao, Chuanxin Lan, Chenxi Wang, Lei Wang, Guoxin Kang, Zhengxin Yang, Yunyou Huang, Xuehai Hong, and Jianfeng Zhan. Beyond benchmarks: Toward causally faithful evaluation of large language models. In *Forty-third International Conference on Machine Learning*.

[36] Xiaorui Wang, Fanda Fan, Chenxi Wang, Yuxuan Yang, Rui Tang, Kuoyu Gao, simiao pang, Yuanfeng Shang, Zhipeng Liu, Wanling Gao, Lei Wang, and Jianfeng Zhan. CombinationTS: A modular framework for understanding time-series forecasting models. In *Forty-third International Conference on Machine Learning*, 2026.

[37] Il'ya Meerovich Sobol'. On sensitivity estimation for nonlinear mathematical models. *Matematicheskoe modelirovanie*, 2(1):112–118, 1990.

[38] Herbert B Enderton. *Elements of set theory*. Gulf Professional Publishing, 1977.

[39] Peter Young, Alice Lai, Micah Hodosh, and Julia Hockenmaier. From image descriptions to visual denotations: New similarity metrics for semantic inference over event descriptions. *Transactions of the Association for Computational Linguistics*, 2:67–78, 2014.

[40] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *Computer Vision – ECCV 2014*, pages 740–755. Springer International Publishing, 2014.

[41] Fuxiao Liu, Yinghan Wang, Tianlu Wang, and Vicente Ordonez. VisualNews: Benchmark and challenges in news image captioning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021.

[42] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. Searching in one billion vectors: Re-rank with source coding. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 861–864, 2011.

[43] Aude Oliva and Antonio Torralba. Modeling the shape of the scene: A holistic representation of the spatial envelope. *International journal of computer vision*, 42(3):145–175, 2001.

[44] Xinyu Zhang, Nandan Thakur, Odunayo Ogundepo, Ehsan Kamalloo, David Alfonso-Hermelo, Xiaoguang Li, Qun Liu, Mehdi Rezagholizadeh, and Jimmy Lin. Miracl: A multilingual retrieval dataset covering 18 diverse languages. *Transactions of the Association for Computational Linguistics*, 11:1114–1131, 2023.

[45] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. Colbertv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734, 2022.

[46] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663*, 2021.

[47] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pages 7534–7550, 2020.

[48] Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. Www'18 open challenge: financial opinion mining and question answering. 2018.

[49] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pages 39–48, 2020.

[50] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. Plaid: an efficient engine for late interaction retrieval. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pages 1747–1756, 2022.

A Notation and Definitions

This appendix defines the evaluation condition space, individual-object attribution metrics, and cross-object interaction analysis used in Section 3.2. Table 5 summarizes the notation.

Table 5: Main notation used in VecDBLens.

Notation	Definition
d	Dataset instance specifying the retrieval corpus and query set.
$\mathrm{corpus}(d), \mathcal{Q}_d, q$	Corpus and query set for d, and a query $q \in \mathcal{Q}_d$.
dim	Vector dimensionality.
e, a	Embedding model and index algorithm.
β, γ	Index construction and query-processing configurations.
$E_e, I_{a,\beta}, S_\gamma$	Embedding, index construction, and query-processing functions.
k	Retrieval cutoff.
eo, ec	Evaluated object instance and evaluation condition instance.
$\Omega_{\neg \mathrm{eo}}$	Compatible condition space for EO eo.
$\mathcal{S}, \mathcal{C}_s^{\mathrm{eo}}$	Strata and conditions evaluated for eo in stratum s.
$M(\mathrm{eo}, \mathrm{ec})$	Performance metric measured under evaluation condition instance ec.
$\mathrm{mean}_M, \mathrm{Var}_M$	Stratified mean and average within-stratum variance.
$\hat{\Delta}_{u-v}$	Mean performance difference between EOs u and v under shared conditions.
$R_k(\mathrm{eo}, \mathrm{ec}), U(e, d, k)$	Evaluated recall and the corresponding Flat recall.
E, I, S	Embedding, construction, and query-processing factors in the interaction analysis.
$y_{\mathrm{ec}}, f_{u,\mathrm{ec}}$	Grid response and effect associated with factor subset u.
$V_{u,\mathrm{ec}}, \rho_u$	Effect variance within evaluation condition instance ec and its aggregated variance share.

Evaluation condition space. Let $\mathcal{G}$ contain the evaluation factors: dataset instance (specifying the corpus and query set), embedding, index algorithm, construction configuration, query-processing configuration, retrieval cutoff, workload, and runtime environment. Each factor g has a set of admissible values Ω_g. For an EO eo, let $\mathcal{G}_{\mathrm{eo}}$ denote the factors specified by that object. Its condition space is

$$\Omega_{\neg \mathrm{eo}} = \left\{ \mathrm{ec} \in \prod_{g \in \mathcal{G} \setminus \mathcal{G}_{\mathrm{eo}}} \Omega_g : (\mathrm{eo}, \mathrm{ec}) \text{ forms a compatible retrieval configuration} \right\}. \tag{3}$$

An embedding EO specifies the embedding factor; an index-configuration EO may specify both the algorithm and its construction settings. Index-specific query-processing mechanisms are evaluated within their corresponding index families. In Experiment 1, the dataset instance d is one part of the evaluation condition. Each observation $M(\mathrm{eo}, \mathrm{ec})$ is measured from the resulting complete configuration. Algorithm-level efficiency comparisons share the framework, hardware, and thread settings.

A.1 Formal Definitions of Attribution Metrics and Interactions

A.1.1 Individual Object Attribution

Mean performance and stability. Partition the evaluated conditions into strata $\mathcal{S}$, such as datasets or filter-ratio groups. For nonempty condition sets $\mathcal{C}_s^{\mathrm{eo}}$, define

$$\mathrm{mean}_M(\mathrm{eo}) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \frac{1}{|\mathcal{C}_s^{\mathrm{eo}}|} \sum_{\mathrm{ec} \in \mathcal{C}_s^{\mathrm{eo}}} M(\mathrm{eo}, \mathrm{ec}), \tag{4}$$

$$\mathrm{Var}_M(\mathrm{eo}) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \mathrm{Var}(\{M(\mathrm{eo}, \mathrm{ec}) : \mathrm{ec} \in \mathcal{C}_s^{\mathrm{eo}}\}). \tag{5}$$

These weights give each stratum equal influence. The first quantity summarizes performance; the second summarizes within-stratum sensitivity to ECs. For positive QPS observations, the geometric mean is obtained by applying the same weights to $\log_2(\mathrm{QPS})$ and exponentiating:

$$\mathrm{GM}_{\mathrm{QPS}}(\mathrm{eo}) = 2^{\mathrm{mean}_{\log_2(\mathrm{QPS})}(\mathrm{eo})}. \tag{6}$$

Differences under shared conditions. For EOs u and v, let $\mathcal{C}_s^{uv} = \mathcal{C}_s^u \cap \mathcal{C}_s^v$ and let $\mathcal{S}_{uv}$ contain the strata with nonempty intersections. Their mean difference is

$$\hat{\Delta}_{u-v} = \frac{1}{|\mathcal{S}_{uv}|} \sum_{s \in \mathcal{S}_{uv}} \frac{1}{|\mathcal{C}_s^{uv}|} \sum_{\mathrm{ec} \in \mathcal{C}_s^{uv}} [M(u, \mathrm{ec}) - M(v, \mathrm{ec})]. \tag{7}$$

The comparison uses the same ECs for both objects and retains equal stratum weights. Positive differences favor u for recall and QPS; negative differences favor u for latency and resource costs.

Exact-search reference. For embedding model instance e and dataset instance d, $U(e, d, k)$ is Flat Recall@k on $\mathrm{corpus}(d)$ and $\mathcal{Q}_d$, using the same similarity function and relevance judgments as the ANN evaluation. Cross-modal retrieval uses dataset-provided relevance annotations; filtered ANN uses exact neighbors within the filter-compatible candidate set. For an ANN configuration eo using embedding model instance e, let d be the dataset instance specified by ec. Define

$$\mathrm{Gap}(\mathrm{eo}, \mathrm{ec}, k) = U(e, d, k) - R_k(\mathrm{eo}, \mathrm{ec}), \tag{8}$$

$$\mathrm{Retention}(\mathrm{eo}, \mathrm{ec}, k) = \frac{R_k(\mathrm{eo}, \mathrm{ec})}{U(e, d, k)}, \qquad U(e, d, k) > 0. \tag{9}$$

Flat recall measures retrieval quality under exact search. Gap and Retention describe the change relative to this reference. For a benchmark with a test5k and a full candidate pool, the two data instances are written explicitly below. For dataset instances d_{test5k} and d_{full} with the same query set and different candidate pools, Pool-Scaling Retention is

$$\mathrm{PSR}(e, d_{\mathrm{test5k}}, d_{\mathrm{full}}, k) = \frac{U(e, d_{\mathrm{full}}, k)}{U(e, d_{\mathrm{test5k}}, k)}, \qquad U(e, d_{\mathrm{test5k}}, k) > 0. \tag{10}$$

A.1.2 Cross-Object Interactions

Matched grids and weights. Within each index family, ec specifies the settings held fixed across the grid, including the dataset instance and runtime environment. The factors E, I, and S represent embedding choice, construction configuration, and query-processing configuration. Their levels form a complete compatible grid

$$\mathcal{X}_{\mathrm{ec}} = \mathcal{E}_{\mathrm{ec}} \times \mathcal{B}_{\mathrm{ec}} \times \Gamma_{\mathrm{ec}}, \qquad w_{\mathrm{ec}}(e, \beta, \gamma) = \frac{1}{|\mathcal{E}_{\mathrm{ec}}|\,|\mathcal{B}_{\mathrm{ec}}|\,|\Gamma_{\mathrm{ec}}|}. \tag{11}$$

The dataset, candidate pool, query set, index algorithm, and runtime environment are fixed within each grid. Construction and query-processing levels are configurations of the corresponding index family, including any coupled parameter settings. The response $y_{\mathrm{ec}}(e, \beta, \gamma)$ is either Retention@10 or $\log_2(\mathrm{QPS})$, analyzed separately. Expectations below use the product weights in Equation 11.

Effect decomposition. Functional ANOVA decomposes the response as

$$\begin{aligned} y_{\mathrm{ec}}(e, \beta, \gamma) = {} & \mathrm{mean}_{\mathrm{ec}}(y) + f_{E,\mathrm{ec}}(e) + f_{I,\mathrm{ec}}(\beta) + f_{S,\mathrm{ec}}(\gamma) \\ & + f_{EI,\mathrm{ec}}(e, \beta) + f_{ES,\mathrm{ec}}(e, \gamma) + f_{IS,\mathrm{ec}}(\beta, \gamma) \\ & + f_{EIS,\mathrm{ec}}(e, \beta, \gamma). \end{aligned} \tag{12}$$

Here $\mathrm{mean}_{\mathrm{ec}}(y) = \mathbb{E}[y_{\mathrm{ec}}]$ is the mean over the factor levels in the grid for ec. For a nonempty subset $u \subseteq \{E, I, S\}$, let x_u denote its factor levels and $-u$ the remaining factors. The effects are defined recursively by

$$f_{u,\mathrm{ec}}(x_u) = \mathbb{E}_{-u}[y_{\mathrm{ec}} \mid x_u] - \mathrm{mean}_{\mathrm{ec}}(y) - \sum_{\emptyset \neq v \subsetneq u} f_{v,\mathrm{ec}}(x_v). \tag{13}$$

For example,

$$f_{E,\mathrm{ec}}(e) = \mathbb{E}_{I,S}[y_{\mathrm{ec}} \mid e] - \mathrm{mean}_{\mathrm{ec}}(y), \tag{14}$$

$$f_{ES,\mathrm{ec}}(e, \gamma) = \mathbb{E}_I[y_{\mathrm{ec}} \mid e, \gamma] - \mathrm{mean}_{\mathrm{ec}}(y) - f_{E,\mathrm{ec}}(e) - f_{S,\mathrm{ec}}(\gamma). \tag{15}$$

Thus, $f_{ES,\mathrm{ec}}$ measures how embedding and query processing jointly affect the response beyond their additive main effects. The three-factor term is the residual after subtracting all main and two-factor effects.

Variance shares. The effect terms have zero mean and are orthogonal under the product weights. Their variance contributions satisfy

$$V_{u,\mathrm{ec}} = \mathbb{E}[f_{u,\mathrm{ec}}^2], \qquad \mathrm{Var}[y_{\mathrm{ec}}] = \sum_{\emptyset \neq u \subseteq \{E,I,S\}} V_{u,\mathrm{ec}}. \tag{16}$$

Let $\mathcal{C}$ denote the set of evaluated EC instances. With equal weights across conditions,

$$\rho_u = \frac{\sum_{\mathrm{ec}\in\mathcal{C}} V_{u,\mathrm{ec}}}{\sum_{\mathrm{ec}\in\mathcal{C}} \mathrm{Var}[y_{\mathrm{ec}}]}. \tag{17}$$

This ratio pools variance contributions before normalization, for a positive total variance. The total interaction share is $\rho_{EI} + \rho_{ES} + \rho_{IS} + \rho_{EIS}$. The decomposition is applied separately to each build repetition, matching the per-repetition results reported in Section 4.2.5.

Conditional interpretation. Variance shares quantify interaction strength, while conditional differences describe its direction. For construction configurations β_1 and β_2, define

$$D_{\mathrm{ec}}(e,\gamma) = y_{\mathrm{ec}}(e,\beta_2,\gamma) - y_{\mathrm{ec}}(e,\beta_1,\gamma). \tag{18}$$

Changes in $D_{\mathrm{ec}}(e,\gamma)$ across search configurations show how query processing modifies the construction effect. A decrease in its absolute value indicates that the construction-related performance difference narrows at that search setting.

B Experimental Configurations

This appendix describes the configurations for the two primary retrieval scenarios in Section 4.

B.1 Runtime Environment

The primary multimodal measurements use a server equipped with two Intel Xeon Gold 6330 CPUs and four Tesla V100-PCIE-32GB GPUs. These measurements use 28 CPU threads or one V100 by default. QPS variance shares in the interaction analysis are computed separately within each run. The VisualNews protocol for Qwen3-VL uses four separately built CAGRA graphs on V100 GPUs and merges their retrieval results.

In the filtered ANN comparisons, Flat, RangePQ, DiGRA, and ACORN-1 use one search thread. ACORN-28 uses 28 search threads. Comparisons of ACORN-28 with Flat therefore describe complete system configurations.

B.2 Multimodal Retrieval Configurations

The multimodal retrieval experiments evaluate text-to-image retrieval on Flickr30K, MSCOCO, and VisualNews. Flickr30K and MSCOCO include both small candidate-pool and full-pool settings, while VisualNews provides a million-scale candidate pool. The evaluated embedding models are CLIP, SigLIP2, and Qwen3-VL, with embedding dimensions of 512, 768, and 4096, respectively. Flat exact search is used as the exact-search reference under each embedding space and similarity metric. The evaluated ANN backends include Faiss HNSW, Faiss IVFPQ, cuVS IVFPQ, and cuVS CAGRA.

Measured configurations. Table 6 lists parameter levels observed in the published multimodal recall results. The interaction analysis uses the complete matched grids within these records: 330 Faiss HNSW and 165 Faiss IVFPQ configurations across five conditions, with each build repetition analyzed separately.

Both IVFPQ implementations use 8 bits per subquantizer. CAGRA uses IVFPQ for graph construction.

B.3 Filtered ANN Configurations

The filtered ANN experiments evaluate ANN search under structured filtering conditions. We use 990,000 base vectors and consider two vector dimensionalities, 128 and 960, instantiated from

Table 6: Observed parameter levels by candidate pool. The two MSCOCO pools have the same marginal levels. PQ subquantizer counts are listed in model-specific rows.

Index	Parameter	Flickr30K test5k	Flickr30K full	MSCOCO (both)	VisualNews
Faiss HNSW	M	16, 32	16, 32	16, 32, 64	32, 48, 64
	`efConstruction`	128, 256	128, 256	200, 400	400, 800
	`efSearch`	32, 64, 128, 256	32, 64, 128, 256	64, 128, 256, 512	128, 256, 512, 1024, 2048
Faiss IVFPQ	`nlist`	256, 512	256, 512, 1024	1024, 2048, 4096	4096, 8192, 16384
	`nprobe`	4, 8, 16, 32	4, 8, 16, 32	8, 16, 32, 64, 128	32, 64, 128, 256, 512
	Subquantizers (CLIP)	32, 64	32, 64	32, 64	64, 128
	Subquantizers (SigLIP2)	32, 64, 96	32, 64, 96	32, 64, 96	64, 96, 128
	Subquantizers (Qwen3-VL)	64, 128, 256, 512	64, 128, 256, 512	64, 128, 256, 512	128, 256, 512
cuVS IVFPQ	`n_lists`	256, 512	256, 512, 1024	1024, 2048, 4096	4096, 8192, 16384
	`n_probes`	4, 8, 16, 32	4, 8, 16, 32	8, 16, 32, 64, 128	32, 64, 128, 256, 512
	Subquantizers (CLIP)	32, 64	32, 64	32, 64	64, 128
	Subquantizers (SigLIP2)	32, 64, 96	32, 64, 96	32, 64, 96	64, 96, 128
	Subquantizers (Qwen3-VL)	64, 128, 256, 512	64, 128, 256, 512	64, 128, 256, 512	128
cuVS CAGRA	Graph degree	32, 64	32, 64	32, 64, 128	64, 96, 128
	Intermediate degree	64, 128	64, 128	64, 128, 255	128, 255
	`itopk_size`	32, 64, 128, 256	32, 64, 128, 256	64, 128, 256, 512	128, 256, 512, 1024
	Search width	1, 2	1, 2	1, 2, 4	1, 2, 4, 8

SIFT1M- and GIST1M-derived workloads augmented with scalar attributes. The filter ratios are 0.01%, 0.1%, 1%, 10%, and 50%, covering workload regimes from highly selective filtering to loose filtering. The query set contains 10,000 queries.

The evaluated methods include Flat exact pre-filtering, RangePQ, DiGRA, and ACORN. Flat exact pre-filtering first applies the structured filter and then performs exact search within the filtered candidate set.

Table 7: Parameter grid for RangePQ.

Parameter	Values
m	$4, 8, 16, 32$
`nbits`	8
`nlist`	$512, 1024$
`cz`	$1024, 2048$
`iter`	200
L	$2500, 4000, 6000, 8000, 10000, 12000, 16000, 20000, 24000, 32000$

Table 8: ACORN construction grid and ACORN-28 search budgets. The ACORN-1 grid includes additional search levels ranging from 10 to 1700.

Parameter	Values
N	990000
M	$32, 64, 128$
M_β	$2M$
`efConstruction`	$100, 400$
`efSearch` (ACORN-28)	$32, 48, 64, 80, 100, 128, 160, 200, 256, 320, 400, 512, 640, 800, 1000, 1200, 1600$
$\gamma_{\text{ACORN}}, r = 0.01\%$	$200, 400, 1000$
$\gamma_{\text{ACORN}}, r = 0.1\%$	$200, 400$
$\gamma_{\text{ACORN}}, r = 1\%$	$50, 120$
$\gamma_{\text{ACORN}}, r = 10\%$	$8, 20$
$\gamma_{\text{ACORN}}, r = 50\%$	$2, 6$

C Multimodal Retrieval Results

C.1 Embedding Dimensionality Controls and Significance Tests

We control output dimensionality to examine its contribution to differences in embedding-based retrieval quality, and assess these differences using matched-query significance tests. All measurements use Flat exact search, keeping index approximation out of the comparison.

Table 9: Parameter grid for DiGRA.

Parameter	Values
`BASE_NUM`	990000
`QUERY_NUM`	10000
`dim`	$128, 960$
`TOPK`	100
M	$32, 64, 128$
`ef_con`	$100, 400$
`ef`	$100, 128, 160, 200, 256, 320, 400, 512, 640, 800, 1000, 1200, 1600$

Table 10: Flat Recall@10 under native and matched output dimensions.

	Mean Flat Recall@10			Largest Holm-adjusted p	
Setting	**CLIP**	**SigLIP2**	**Qwen3-VL**	**Permutation**	**Cluster-robust t**
Native dimensions	0.5345	0.2249	0.7235	1.50×10^{-4}	6.42×10^{-10}
PCA-512	0.5345	0.1252	0.7213	1.50×10^{-4}	5.81×10^{-10}
First-512 truncation	0.5345	0.1782	0.7065	1.50×10^{-4}	5.54×10^{-7}

Matched-dimension controls. CLIP, SigLIP2, and Qwen3-VL produce 512-, 768-, and 4,096-dimensional embeddings, respectively. We apply two post-hoc controls to SigLIP2 and Qwen3-VL: PCA-512, fitted using corpus vectors only and applied to both corpus and query vectors, and first-512 truncation, which retains the first 512 coordinates. Both transformations are followed by L_2 normalization; CLIP remains unchanged at its native 512 dimensions.

The comparisons cover the Flickr30K and MSCOCO test5k and full-pool conditions, plus VisualNews. Table 10 reports the five-condition mean Flat Recall@10 under native dimensions, PCA-512, and first-512 truncation. All three settings use the same 1,446-query VisualNews subset shared by the models.

Significance tests. For each model pair within each condition, we compute the difference between the two binary Flat Recall@10 outcomes for every query. Queries sharing the same relevant-image set form one cluster, giving 1,000 clusters per Flickr30K condition, 5,126 per MSCOCO condition, and 1,446 for VisualNews. The primary analysis uses a two-sided cluster-level sign-flip permutation test with 100,000 permutations and 10,000 cluster-bootstrap resamples for 95% confidence intervals. Entire clusters are flipped or resampled together, preserving the query-weighted mean difference. We additionally use a two-sided paired t-test implemented as an intercept-only regression on query-level differences with CR1 cluster-robust standard errors and $G - 1$ degrees of freedom, where G is the number of clusters. Holm correction is applied to the 15 model-pair-by-condition comparisons separately within each dimensionality setting and test method.

Results. Table 10 shows that model differences remain statistically significant when output dimensionality is matched. All 15 comparisons in each setting are significant under both tests after Holm correction, and all cluster-bootstrap and cluster-robust 95% confidence intervals exclude zero. The permutation estimates reach the Monte Carlo resolution of $1/(100{,}000 + 1)$ before correction. The two dimensionality controls also affect the models differently: Qwen3-VL's mean Recall@10 changes from 0.7235 to 0.7213 under PCA and 0.7065 under truncation, while SigLIP2's changes from 0.2249 to 0.1252 and 0.1782.

C.2 Candidate-pool scaling and ANN retention

Table 11 reports the ratio of full-pool to test5k Flat Recall@10. The input recalls are given in Table 1. Ratios are computed from unrounded recall values and averaged with equal dataset weights before rounding for display.

Table 12 distinguishes average performance over the configuration grid from the best performance observed within each condition. Mean results first average configurations within each observed dataset-pool–embedding condition and then give each observed condition equal weight. The variance column summarizes variability across condition means. Best-configuration Retention selects the highest Retention within each condition before macro-averaging.

Table 11: Pool-Scaling Retention (PSR) at cutoff 10.

Embedding	Flickr30K	MSCOCO	Mean PSR
Qwen3-VL	77.7%	66.9%	72.3%
CLIP	54.9%	44.2%	49.6%
SigLIP2	37.5%	28.9%	33.2%

Table 12: ANN Retention@10 and Gap@10 relative to the condition-specific Flat reference.

Backend	Mean Retention	Between-condition variance	95% CI	Best Retention	Mean Gap
cuVS CAGRA	0.9959	0.000028	[0.9930, 0.9983]	1.0000	0.0012
Faiss HNSW	0.9875	0.000201	[0.9801, 0.9938]	0.9985	0.0041
Faiss IVFPQ	0.7932	0.015712	[0.7319, 0.8538]	0.9083	0.0751
cuVS IVFPQ	0.7409	0.012833	[0.6853, 0.7958]	0.8726	0.1015

C.3 Throughput under matched quality targets

Table 13 reports Flat throughput by condition for CLIP (512 dimensions), SigLIP2 (768 dimensions), and Qwen3-VL (4096 dimensions). Repeated Flat QPS observations within the same condition are averaged. Ratios are computed from unrounded values.

Table 13: Flat QPS for the three embedding models.

Condition	CLIP	SigLIP2	Qwen3-VL	CLIP/Qwen3-VL
Flickr30K test5k	9,305.00	7,718.89	1,662.53	5.60×
Flickr30K full-pool	568.94	381.01	61.64	9.23×
MSCOCO test5k	3,491.97	2,373.31	348.16	10.03×
MSCOCO full-pool	207.18	139.68	24.94	8.31×
VisualNews	13.81	9.22	1.61	8.55×

Table 14 lists the highest-throughput measured configuration of each backend on VisualNews with CLIP, subject to $\mathrm{Retention@10} \geq 0.99$. Here efC and efS denote `efConstruction` and `efSearch`. The throughput measurements use a separate VisualNews candidate pool, with Retention computed against its corresponding Flat reference.

Table 14: Highest-throughput configurations satisfying $\mathrm{Retention@10} \geq 0.99$ on VisualNews with CLIP.

Backend	Recall@10	Retention@10	QPS	Resources	Parameters
Flat	0.29895	1.00000	13.81	CPU, 28 threads	Exact search
Faiss HNSW	0.29605	0.99030	1,898.56	CPU, 28 threads	$M = 32$, efC $= 400$, efS $= 512$
cuVS CAGRA	0.29680	0.99281	29,433.58	One V100	Graph degree $= 64$, intermediate degree $= 128$, itopk size $= 256$, search width $= 2$

D Filtered ANN Results

D.1 Real workloads and random seed settings

Real workloads. We evaluate ACORN on CC-News (620,643 vectors) with 3-day and 30-day timestamp windows and on App Reviews (277,936 vectors) with conjunctions of three attribute-range predicates. Each workload uses 10,000 queries, 384-dimensional vectors, and exact filtered-neighbor IDs. We evaluate 156 configurations per workload using $M \in \{32, 64, 128\}$, $M_\beta = 2M$, $\gamma_{\mathrm{ACORN}} \in \{2, 6\}$, `efConstruction`$\in \{100, 400\}$, and 13 search budgets from 100 to 1600, with 28 search threads.

Table 15: ACORN construction-parameter audits on real workloads. Drops are absolute decreases in Recall@100 under matched ECs. The M comparisons jointly change $M_\beta = 2M$.

Workload	Parameter	Comparisons	Drops > 0	Drops $> 10^{-4}$	Max drop
CC-News, 3-day	M	104	8	8	0.0134
	`efConstruction`	78	39	39	0.3107
CC-News, 30-day	M	104	15	14	0.0115
	`efConstruction`	78	39	39	0.3380
App Reviews	M	104	24	24	0.0887
	`efConstruction`	78	35	35	0.3061

Increasing `efConstruction` from 100 to 400 lowers Recall@100 in 39 of 78 matched comparisons for each CC-News window and in 35 of 78 on App Reviews (Table 15).

Random seed settings. The ACORN graph construction seed for both CC-News windows was 12345. The App Reviews driver initialized the C random number generator with `srand(0)`. Each real-workload configuration used one index build. For the SIFT128 graph analysis, we built each of the four ACORN configurations twice using the same graph construction seed (12345) and 112 construction threads.

D.2 Throughput and construction cost

Table 16 reports the highest QPS at $\mathrm{Recall@100} \geq 0.95$ for each dimensionality, filter ratio, and method; a dash indicates no qualifying configuration. ACORN-28 uses 28 search threads; others use one. Speedup is the ACORN-28/Flat QPS ratio. On 128-dimensional SIFT, ACORN-28 achieves a median $14.243\times$ QPS speedup over ACORN-1 across 1,122 pairs matched on filter ratio, construction parameters, and search budget.

Table 16: Highest measured QPS subject to $\mathrm{Recall@100} \geq 0.95$.

dim	Filter ratio	Flat	RangePQ	DiGRA	ACORN-1	ACORN-28	Speedup
128	0.01%	8,949.74	4,483.70	4,238.16	—	—	—
128	0.1%	3,913.89	—	2,609.59	77.80	1,416.40	0.4×
128	1%	537.43	—	1,789.91	431.20	5,821.10	10.8×
128	10%	48.07	—	1,592.11	687.50	9,678.30	201.3×
128	50%	9.87	—	1,348.07	610.40	10,377.60	1,051.7×
960	0.01%	1,158.62	—	3,843.01	—	—	—
960	0.1%	471.99	—	2,246.98	—	—	—
960	1%	32.41	—	1,250.22	35.70	3,157.80	97.4×
960	10%	2.83	—	1,200.50	23.20	1,767.30	624.5×
960	50%	0.53	—	904.78	15.20	977.80	1,842.9×

Beyond query throughput, Table 17 reports construction time and peak RSS across construction configurations. DiGRA requires substantially longer construction times than RangePQ and ACORN-1. ACORN-1 has the lowest median construction time, while RangePQ has the lowest median peak RSS among the reported methods at both dimensionalities.

Table 17: Construction time (s) and peak RSS (MB) by method and dimensionality.

dim	Method	Configs.	Median time	Time range	Median RSS	RSS range
128	RangePQ	80	21.49	5.87–111.61	1,136.3	1,085.8–1,594.0
128	DiGRA	30	6,162.48	2,291.21–14,852.00	6,106.1	3,689.1–10,940.3
128	ACORN-1	66	3.71	1.44–7.90	4,426.3	1,877.0–12,004.1
960	RangePQ	80	27.49	8.73–125.88	8,894.0	7,143.1–11,496.6
960	DiGRA	30	7,772.06	2,879.72–19,788.00	9,280.2	6,863.1–14,114.3
960	ACORN-1	60	10.75	6.03–36.55	10,523.4	8,129.5–13,759.5

E Additional Retrieval Scenarios

We extend VecDBLens to Chinese text retrieval and multi-vector retrieval. Task quality is measured using dataset relevance judgments. ANN@100 measures recovery of the corresponding exact top-100 results.

E.1 Chinese Text Retrieval

We evaluate Qwen3-VL (4096 dimensions) and SigLIP2 (768 dimensions) on MIRACL Chinese [44], using 4,934,368 passages and 393 development queries. Embeddings are normalized. Each model is evaluated with Flat, five HNSW search budgets (`efSearch` $= 128, 256, 512, 1024, 2048$), and 45 IVFPQ configurations. The IVFPQ grid combines `nlist` $\in \{4096, 8192, 16384\}$, `nprobe` $\in \{32, 64, 128, 256, 512\}$, and three subquantizer counts: $\{64, 96, 128\}$ for SigLIP2 and $\{128, 256, 512\}$ for Qwen3-VL, with 8 bits per subquantizer. QPS is the median of three full-query-batch measurements using 28 CPU threads, excluding encoding.

Table 18 separates model-dependent task quality from ANN approximation. HNSW preserves each model's task metrics at the displayed precision across search budgets, while IVFPQ introduces additional degradation. High exact-neighbor recovery therefore does not compensate for low representation quality. The model comparison includes their encoding settings: SigLIP2's 64-token limit truncates approximately 56.55% of the corpus.

Table 18: MIRACL Chinese results. Ranges give the minimum and maximum of each metric across all tested configurations; extrema need not occur at the same configuration.

Model	Method	nDCG@10	Recall@100	ANN@100	QPS
SigLIP2	Flat	0.0057	0.0217	1.0000	247.2
SigLIP2	HNSW	0.0057	0.0217	0.9468–0.9936	1060.8–16642.2
SigLIP2	IVFPQ	0.0006–0.0035	0.0061–0.0138	0.3393–0.5678	167.8–6190.4
Qwen3-VL	Flat	0.5973	0.9319	1.0000	44.4
Qwen3-VL	HNSW	0.5973	0.9319	0.9911–0.9992	46.2–2691.1
Qwen3-VL	IVFPQ	0.4819–0.5782	0.7703–0.9107	0.5438–0.7892	58.2–1494.6

Table 19: Multi-vector retrieval at the two endpoints of the evaluated candidate-budget range.

Dataset	Configuration	nDCG@10	Recall@100	ANN@100	Latency (ms)
SciFact	Exact MaxSim	0.6464	0.8942	1.0000	9.89
SciFact	PLAID, $K = 100$	0.6465	0.8909	0.8884	6.69
SciFact	PLAID, $K = 1000$	0.6464	0.8942	0.9953	10.89
FiQA	Exact MaxSim	0.3472	0.6277	1.0000	109.27
FiQA	PLAID, $K = 100$	0.3457	0.6201	0.8825	7.20
FiQA	PLAID, $K = 1000$	0.3456	0.6235	0.9713	11.65

E.2 Multi-Vector Retrieval

We evaluate ColBERTv2 [45] on BEIR SciFact and FiQA [46, 47, 48], with 5,183/57,638 documents and 300/648 test queries, respectively. Token vectors have 128 dimensions, with query/document lengths of 32/180 and the model's masking and normalization. Exact search applies MaxSim [49] to uncompressed token representations over the full corpus. PLAID [50] generates candidates with 2-bit compression and reranks them using the same uncompressed representations, with budgets $K \in \{100, 250, 500, 1000\}$. Measurements use one V100 GPU and 16 CPU threads; latency excludes encoding.

Table 19 reports the exact reference and the smallest and largest evaluated candidate budgets. On FiQA, $K = 100$ reduces mean latency from 109.27 to 7.20 ms, with an nDCG@10 difference of -0.001452 (pointwise 95% query-bootstrap interval: $[-0.004604, 0.000182]$). Larger budgets improve exact-neighbor recovery without consistent gains in task quality. On SciFact, $K = 1000$ is slower than exact search, showing how corpus size and search budget affect the quality–cost trade-off.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction describe VecDBLens as a framework for individual object attribution and cross-object interaction analysis. Section 4 evaluates the framework in two primary retrieval scenarios, with additional scenarios reported in Appendix E.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: We explicitly discuss the limitations of our framework, including its current reliance on system-level metrics rather than fine-grained semantic diagnostics and the specific scope of the evaluated workloads.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not present new theorems or formal proof-based claims. Its mathematical content consists of framework definitions and metrics for individual object attribution and cross-object interaction analysis, with notation and formal definitions provided in the main text and appendix.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper describes the evaluated datasets, embedding models, index backends, filtered ANN methods, parameter grids, evaluation metrics, statistical protocol, hardware, and runtime settings. The appendix further provides full experimental configurations and detailed numerical results supporting the main claims.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The paper provides a publicly accessible code repository link. The experiments are based on public datasets, publicly available embedding models, and open-source or publicly available vector search systems; the appendix and repository provide the configurations needed to reproduce the reported results.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper specifies the retrieval datasets, candidate-pool settings, embedding dimensions, index algorithms, construction and search parameter grids, filter ratios, thread settings, GPU settings, and evaluation metrics. Since the paper evaluates existing embedding models and ANN systems rather than training new models, optimizer details are not applicable.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The paper reports mean performance, variance, and 95% bootstrap confidence intervals. Differences between embedding models are assessed using cluster-aware significance tests with Holm correction for multiple comparisons, as detailed in Appendix C.1.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The paper describes a shared runtime environment used across all experiments, including CPU/GPU hardware, default thread settings, and GPU usage. It also specifies experiment-specific execution details when they differ from the default setup, such as four-GPU graph construction for high-dimensional Qwen3-VL embeddings on VisualNews and separate ACORN-1/ACORN-28 settings for filtered ANN search.

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification:The research is an empirical evaluation study of vector database retrieval systems using public datasets, public or cited embedding models, and open-source or publicly available vector search systems. To the best of our knowledge, the work conforms to the NeurIPS Code of Ethics.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The paper discusses how more transparent vector database evaluation can help practitioners identify true retrieval bottlenecks and avoid misleading performance conclusions.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release high-risk pretrained models, image generators, or newly scraped datasets. It evaluates existing retrieval datasets, embedding models, and vector search systems.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification:The paper cites the datasets, embedding models, ANN libraries, and filtered ANN systems used in the experiments, including their original sources where available. The experiments are conducted using existing public or publicly available assets under their respective access conditions.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce a new dataset or pretrained model as a released asset. The contribution is a modular evaluation and attribution framework together with empirical findings based on existing assets.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification:The paper does not involve crowdsourcing experiments or research with human subjects.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve human subjects, user studies, or crowdsourcing experiments, so IRB approval is not applicable.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification:AI/LLM tools were used only for writing assistance, including language polishing, editing, and formatting. They were not used as an important, original, or non-standard component of the core method, and did not affect the experimental design, implementation, analysis, scientific rigor, or originality of the research.